23CSE455 Design Patterns

Design Pattern Identification and Evolution

in a Laravel-Based Property Rental Application

Team B4

Earlapally Sahith Siddarth	Roll number: AM.SC.U4CSE23217; Batch: 2
Bussa Surya	Roll number: AM.SC.U4CSE23213; Batch: 2
Eati Manoj	Roll number: AM.SC.U4CSE23317; Batch: 2
Kankanala Sanithya Reddy	Roll number: AM.SC.U4CSE23327; Batch: 2

Submission status. The technical report, source snapshots, diagrams, tests, four-language implementations, prompts, CSV files, and Excel workbook are complete.

11 October 2026

Contents

1 Executive summary 1

2 Application description and framework architecture 1

3 Part 1 - Pattern discovery 3

4 Part 2 - Software evolution and refactoring 7

5 Part 3 - Cross-language analysis 11

6 LLM records and human verification 12

7 Dataset, metrics, and audit trail 13

8 Findings and limitations 13

9 Submission readiness 13

10 References 14

A Evidence index 14

B Folder structure 14

1 Executive summary

This case study examines the public MIT-licensed *Property Management* Laravel application by Mohsin Rafique at baseline commit `f127b4e218018715586a52cede08e182ca45c983`. The inspected Laravel 12 source implements property, owner and tenant management, authentication, role authorization, rent/payment receipts, security deposits, and Blade views. Five baseline collaborations are confirmed: Model-View-Controller (MVC), Active Record, Chain of Responsibility, Template Method, and Laravel Facade. Repository, Strategy, and Observer claims were rejected at baseline because the application directly uses Eloquent, declares no interchangeable business algorithm, and has no application notification collaboration.

Three changes evolve that preserved baseline. CH01 introduces Strategy for dynamic pricing and integrates it with rent-receipt creation. CH02 introduces Specification for composable property filters and integrates request-driven filters with the property controller. CH03 introduces State plus an Eloquent model and migration for digital contracts. The cumulative design has eight distinct pattern types and eight pattern instances. Production PHP under `app/` grows from 2,055 to 2,450 physical LOC. The final Laravel suite passes 2/2 and the focused evolution runner passes 10/10. The unchanged upstream baseline has one passing test and one stale example test that expects HTTP 200 even though the root route intentionally redirects to login; this discrepancy is preserved as evidence and corrected only in the evolved snapshot.

The two most significant evolution patterns, Strategy and Specification, are implemented against identical fixtures in Java, Python, JavaScript, and C++. All Strategy variants pass 6/6 checks; all Specification variants pass 5/5. One warmup and five measured processes were run after compilation where applicable. The measured medians include runtime startup and test execution, so they are reproducibility evidence rather than a universal performance ranking.

2 Application description and framework architecture

2.1 Application and source identity

The selected application is *Property Management*, available at `https://github.com/mohsin-rafique/property-management`. The baseline is upstream commit `f127b4e218018715586a52cede08e182ca45c983`, dated 1 September 2026 and accessed 11 October 2026. The repository declares the MIT license. The exact source without Git metadata, dependencies, or environment secrets is preserved at `Part1/code`; the cumulative evolved source is preserved at `Part2/code`.

The public application targets PHP 8.2+ and Laravel 12, uses Eloquent with MySQL/MariaDB, Blade views, Laravel UI authentication, role middleware, and DomPDF. The baseline and final source manifests provide integrity evidence. Local evolution commits preserve each change:

CH01	`7e9bd9ded113e91cca10e727a7eeff29c88d0ed8`
CH02	`2b2dd6246dd50f4a53a991e5429b22da1b1a5c4f`
CH03	`8e2af082067bb83aab7e40caf2b7a463fef8b643`
Verification	`27b78c16c897c845efe052101e7f47f2ad7c25cf`

Faculty approval remains a human-only confirmation and is not fabricated.

2.2 Functional scope

Area	Primary symbols	Observed responsibility

Property management	`Property`, `PropertyController`, Blade views	Create, view, update, delete, paginate, assign owner/tenant, and track vacancy.
Owner management	`Owner`, `OwnerController`	Create owner accounts and expose owner-property relationships.
Tenant management	`Tenant`, `TenantController`, tenant dashboard	Create tenant accounts, assign properties, and expose tenant receipts.
Property search	`PropertyController::index`	Baseline pagination; evolved address, rent, status, and composable domain specifications.
Lease management	Property tenant assignment; no baseline lease entity	Baseline records occupancy but lacks a dedicated lease. CH03 adds digital-contract state and persistence.
Payment	`RentReceipt`, electricity and maintenance receipts	Record rent and utility/maintenance payments and generate PDFs.
Notification	`User::Notifiable`; no application notification workflow	Framework capability is present, but no baseline Observer collaboration is claimed.
Authentication / authorization	Laravel UI controllers, `auth` and `RoleMiddleware`	Authenticate users and restrict tenant, owner, and administrator routes.

2.3 Architecture

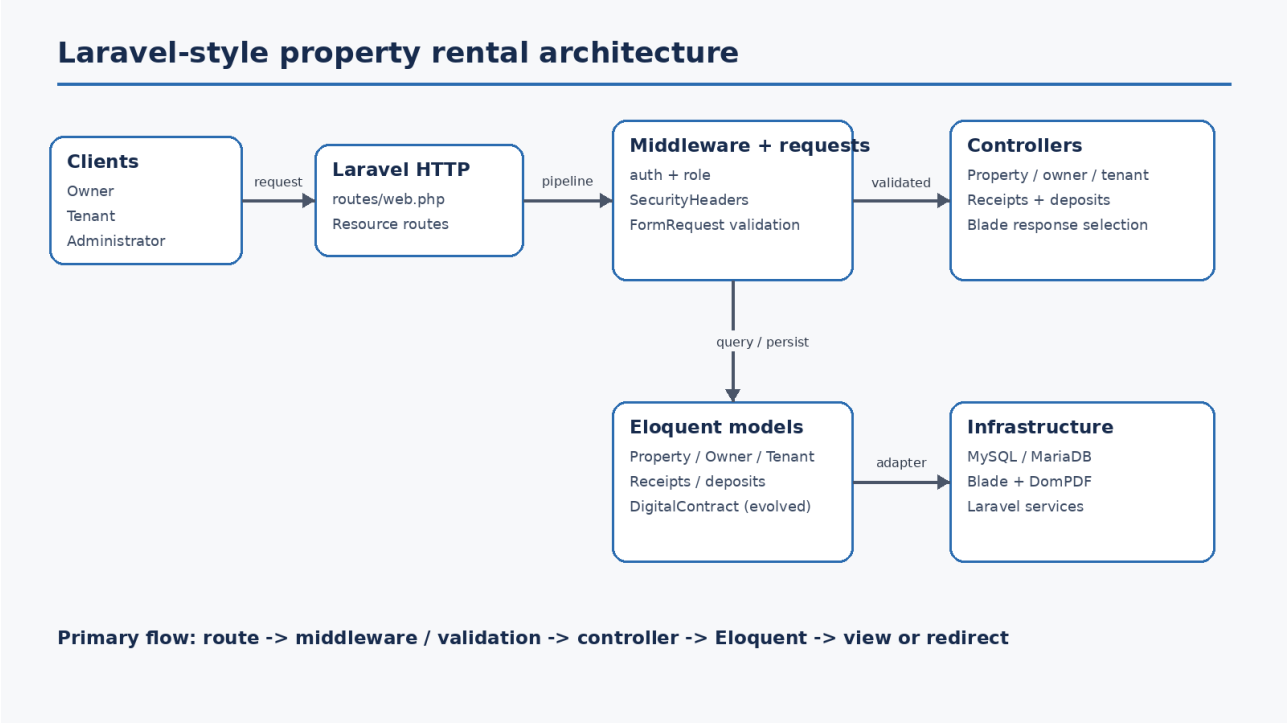


Figure 1: Laravel-style layers and the primary request flow. Editable source: `architecture.puml`.

A request traverses Laravel's global and route middleware, reaches a resource controller, invokes Eloquent Active Record models, and selects a Blade view or redirect response. `RoleMiddleware` can stop unauthorized requests while `SecurityHeaders` decorates downstream responses. CH01 adds a container-bound pricing Strategy, CH02 lets `PropertyController` compose filtering Specifications, and CH03 adds a State-driven contract domain object plus Eloquent persistence.

2.4 Environment and execution

Laravel dependencies were resolved in the `composer:2` container to Laravel 12.53.0; syntax and focused domain checks ran in `php:8.3-cli`. Part 3 used OpenJDK 17.0.20.1, Python 3.12.3, Node.js 24.14.1, and G++ 13.3.0 with C++20. Exact commands, versions, timing rules, and limitations are recorded in `README.md` and `Part3/evidence/environment_versions.txt`.

3 Part 1 - Pattern discovery

3.1 Method

Discovery was performed against the recorded public commit, not README claims alone. Controllers, Eloquent models, middleware, form requests, service-provider configuration, routes, and Blade views were inspected. Each candidate required recognizable participant roles and an observed call or framework lifecycle. Framework-origin participants are named explicitly. Absence is also evidence: direct Eloquent calls reject a Repository claim, the unused `Notifiable` trait does not establish an Observer collaboration, and no baseline algorithm family establishes Strategy.

3.2 Baseline totals

Measure	Baseline
Distinct confirmed pattern types	5
Confirmed pattern instances	5
Concrete participant classes in mapped roles	18
Unique mapped participant symbols	15
Rejected candidates	3
Uncertain candidates	0

Concrete participant counts are per collaboration; unique-symbol counts remove overlaps. The authoritative row-level inventory is `data/patterns.csv`. Count logic and every cumulative snapshot are in `data/pattern_counts.csv`.

3.3 Pattern inventory

Pattern type	Instance	Count	Variable concrete roles
Model-View-Controller	Property Management Request Coordination	1	Resource controller, Eloquent models, Blade views.
Active Record	Property and Rental Record Persistence	1	`Property`, `Owner`, `Tenant`, receipt models.
Chain of Responsibility	Authentication and Security Pipeline	1	`RoleMiddleware`, `SecurityHeaders`, framework auth handler.

Template Method	Form Request Validation Lifecycle	1	Store/update request subclasses override lifecycle hooks.
Facade	Tenant Registration Infrastructure Access	1	Laravel `DB` and `Hash` facades.

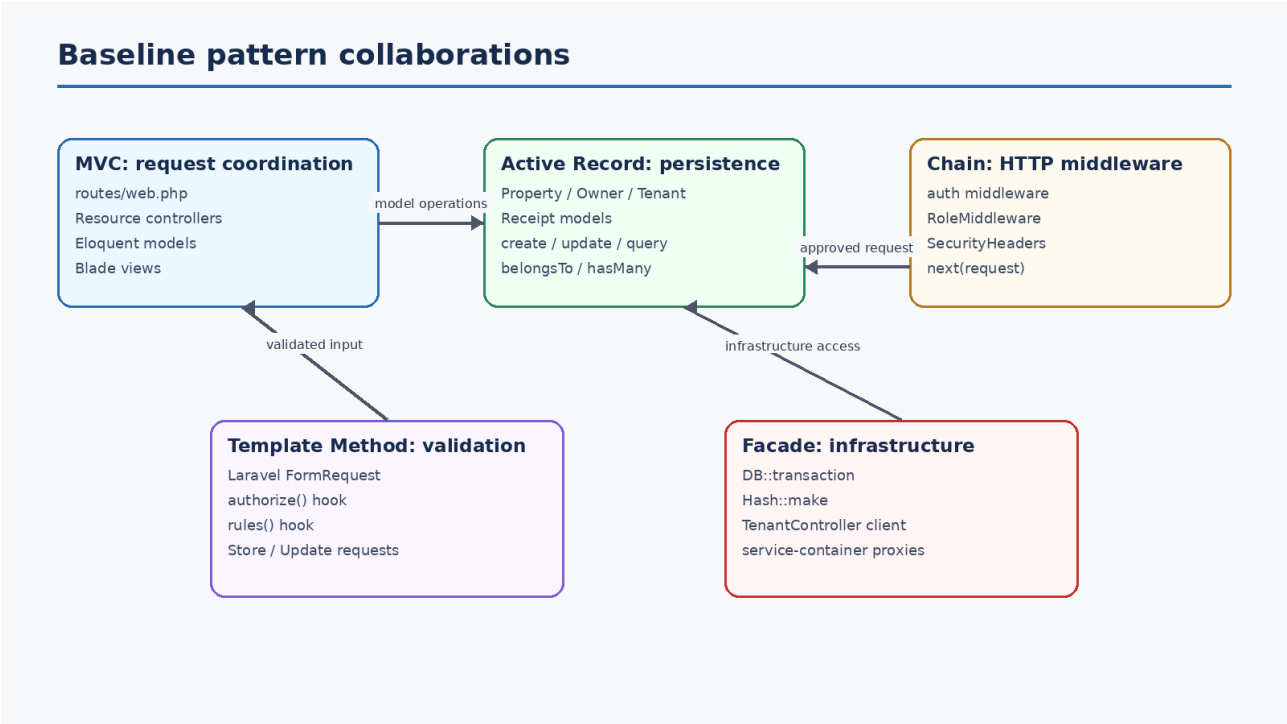


Figure 2: Baseline collaborations. Editable source: `baseline_patterns.puml`.

3.4 Instance 1: Property Management Request Coordination - MVC

Category and origin. Architecture pattern; application structure using Laravel framework conventions.

Participants. Controller = `PropertyController` (lines 11–119). Models = `Property`, `Owner`, and `Tenant`. Views = the four templates under `resources/views/properties`. Router = resource registration in `routes/web.php`.

Evidence. The index action queries the model and selects `properties.index`; create, store, show, edit, update, and destroy follow the same request-controller-model-view separation.

```
1  class PropertyController extends Controller
2  {
3      public function index()
4      {
5          $properties = Property::with(['owner', 'tenant'])
6              ->latest()
7              ->paginate(10);
8
9          return view('properties.index', compact('properties'));
10     }
11
12     public function create()
13     {
14         $owners = Owner::all();
15         // Only show tenants not already assigned to a property
16         $tenants = Tenant::whereDoesntHave('property')->get();
17
18         return view('properties.create', compact('owners', 'tenants'));
```

Verification. The route file, controller actions, Eloquent model, and actual Blade templates jointly confirm the collaboration; no View class is invented.

3.5 Instance 2: Property and Rental Record Persistence - Active Record

Category and origin. Enterprise data-source pattern; Laravel Eloquent framework plus application models.

Participants. Active Records = `Property`, `Owner`, `Tenant`, and receipt models. Mapping base = `Illuminate\Database\Eloquent\Model`. Relationships are declared with `belongsTo` and `hasMany`; controllers call static query and instance persistence methods.

Evidence. `Property` declares fillable attributes, casts, and relationship methods, while `PropertyController` directly calls `Property::create`, query builders, `update`, and `delete`.

```php
class Property extends Model
{
    use HasFactory;

    protected $fillable = [
        'owner_id',
        'tenant_id',
        'address',
        'monthly_rent',
        'maintenance_total',
        'owner_maintenance_percent',
        'tenant_maintenance_percent',
        'electricity_rate_per_unit',
        'status',
    ];

    //  Casts
    // In Yii2, types come from DB schema. In Laravel,
    // $casts tells Eloquent how to convert attributes.
    // When you do $property->monthly_rent, it returns a float, not string.

    protected $casts = [
        'monthly_rent' => 'decimal:2',
        'maintenance_total' => 'decimal:2',
        'electricity_rate_per_unit' => 'decimal:2',
    ];

    public function owner()
    {
        return $this->belongsTo(Owner::class);
    }

    public function tenant()
    {
        return $this->belongsTo(Tenant::class);
    }

    public function rentReceipts()
    {
        return $this->hasMany(RentReceipt::class);
```

Verification. The model inheritance and direct persistence calls confirm Active Record. This same evidence rejects a separate application Repository pattern.

3.6 Instance 3: HTTP Authentication and Security Pipeline - Chain of Responsibility

Category and origin. GoF Behavioral / framework pipeline; Laravel and application middleware.

Participants. Handler = Laravel middleware contract and closure. Concrete handlers = framework `auth`, application `RoleMiddleware`, and `SecurityHeaders`. Client/configuration = route groups in `routes/web.php` and middleware registration in `bootstrap/app.php`.

Evidence. `RoleMiddleware` returns a redirect or 403 to stop the chain, otherwise calls `\$next(\$request)`. `SecurityHeaders` calls the next handler first and then decorates its response.

```
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        if (!auth()->check()) {
            return redirect()->route('login');
        }

        if (!in_array(auth()->user()->role, $roles)) {
            abort(403, 'You do not have permission to access this page.');
        }

        return $next($request);
    }
```

Verification. The route groups show ordered use of `auth` and role checks, while the middleware bodies expose pass/stop behavior.

3.7 Instance 4: Form Request Validation Lifecycle - Template Method

Category and origin. GoF Behavioral / framework lifecycle; Laravel base class with application hook implementations.

Participants. Abstract template = Laravel `FormRequest`. Primitive hook operations = `authorize` and `rules`. Concrete classes include `StorePropertyRequest`, `StoreTenantRequest`, and `StoreRentReceiptReque`

Evidence. Laravel owns resolution, authorization, validation, and controller invocation. Each application subclass supplies only its authorization decision and validation-rule map.

```
class StorePropertyRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'owner_id' => 'required|exists:owners,id',
            'tenant_id' => 'nullable|exists:tenants,id',
            'address' => 'required|string|max:500',
            'monthly_rent' => 'required|numeric|min:0',
            'maintenance_total' => 'nullable|numeric|min:0',
            'owner_maintenance_percent' => 'nullable|integer|min:0|max:100',
            'tenant_maintenance_percent' => 'nullable|integer|min:0|max:100',
            'electricity_rate_per_unit' => 'nullable|numeric|min:0',
        ];
    }
}
```

Verification. Multiple request subclasses repeat the same two hook signatures while controllers receive already validated request objects.

3.8 Instance 5: Tenant Registration Infrastructure Access - Facade

Category and origin. GoF Structural as implemented by Laravel's service-container facades.

Participants. Facades = `DB` and `Hash`. Hidden subsystems = Laravel database manager/connection and hashing manager. Client = `TenantController`.

Evidence. Tenant creation uses `DB::transaction` and `Hash::make` without locating or constructing the underlying services.

```
    public function store(StoreTenantRequest $request)
    {
        DB::transaction(function () use ($request) {
```

```
            $user = User::create([
                'name' => $request->name,
                'email' => $request->email,
                'password' => Hash::make($request->password),
                'role' => 'tenant',
            ]);

            Tenant::create([
                'user_id' => $user->id,
                'name' => $request->name,
                'phone' => $request->phone,
                'cnic' => $request->cnic,
                'address' => $request->address,
            ]);
        });
```

Verification. Imports and static proxy calls map directly to Laravel facade participants; the controller remains the client.

3.9 Rejected candidates

- **Repository**: rejected at baseline because controllers query and mutate Eloquent Active Records directly; no application repository interface or adapter exists.
- **Strategy**: rejected at baseline because no interchangeable application algorithm family and context are declared. CH01 deliberately introduces it.
- **Observer**: rejected at baseline because `User` using Laravel's `Notifiable` trait is capability only; the application registers no event/listener or observer collaboration.

4 Part 2 - Software evolution and refactoring

4.1 Evolution overview

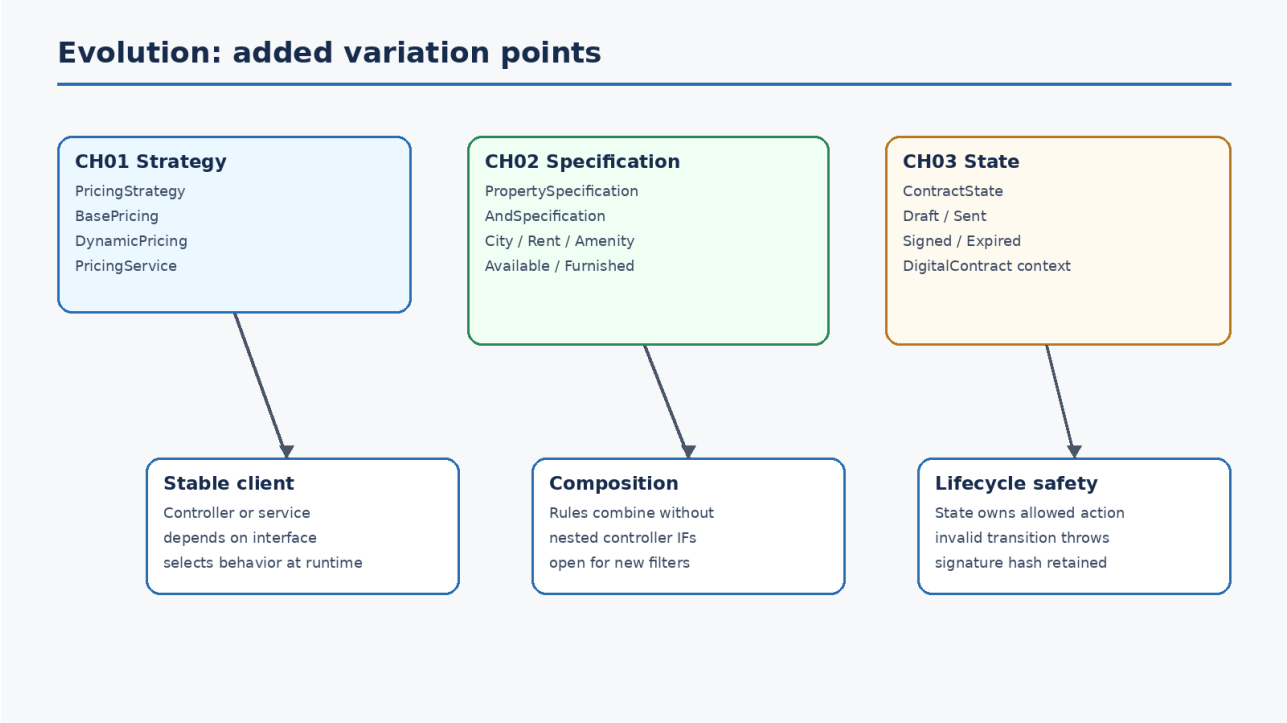


Figure 3: Three new variation points. Editable source: `evolved_patterns.puml`.

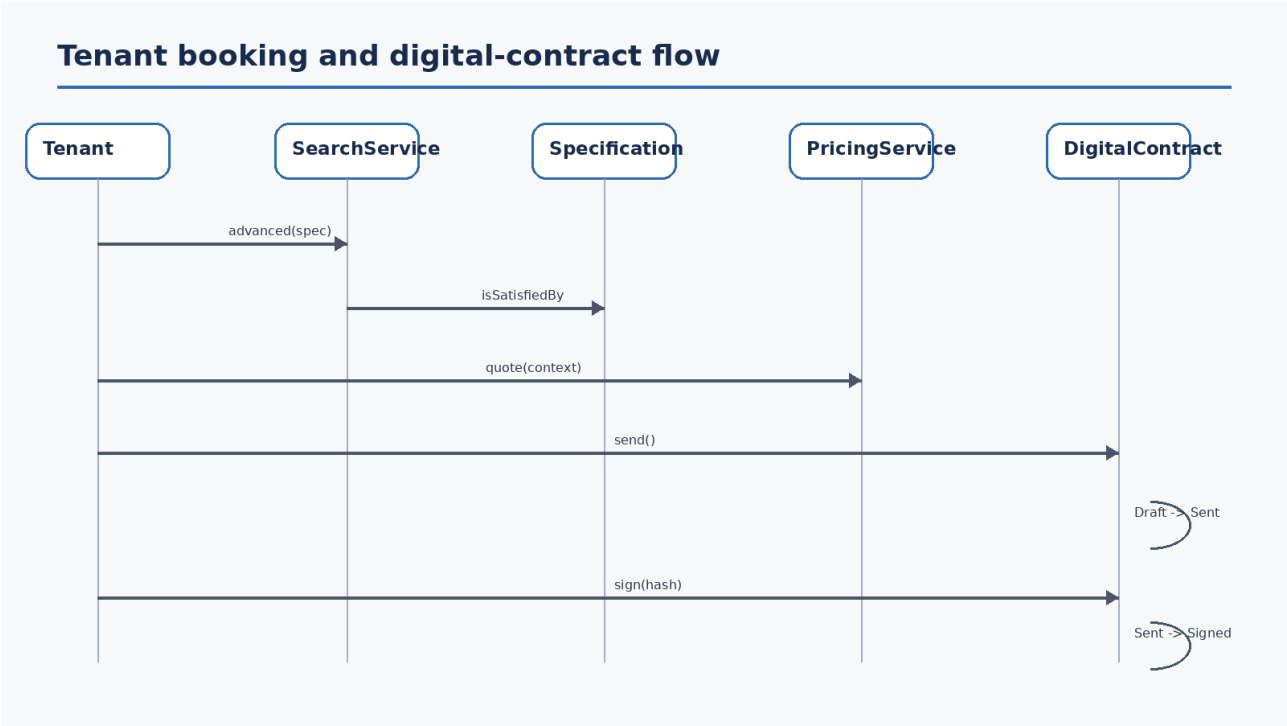


Figure 4: Search, quote, and contract-signing flow. Editable source: `evolution_sequence.puml`.

4.2 CH01 - Dynamic pricing

Requirement and acceptance criteria. The application must select base or dynamic pricing without changing the quoting client. Peak months 4, 5, and 12 add 20 percent; occupancy of at least 0.80 adds 10 percent; leases of at least 12 months subtract 5 percent; adjustments are additive and rounded to two decimals. Invalid month, occupancy, or duration values must fail.

Before. `RentReceiptController::store` accepts a submitted amount directly; there is no pricing variation point.

After. Strategy = `PricingStrategy`. ConcreteStrategy = `BasePricing` and `DynamicPricing`. Context = `PricingService`. Parameter Object = `PricingContext`. `AppServiceProvider` binds the interface to `DynamicPricing`; `RentReceiptController` calls the service when dynamic pricing is requested.

```
    public function quote(float $baseMonthlyRent, PricingContext $context): float
    {
        if ($baseMonthlyRent <= 0) {
            throw new InvalidArgumentException('Base monthly rent must be positive.');
        }

        $multiplier = 1.0;
        $multiplier += in_array($context->month, [4, 5, 12], true) ? 0.20 : 0.0;
        $multiplier += $context->occupancyRate >= 0.80 ? 0.10 : 0.0;
        $multiplier -= $context->leaseMonths >= 12 ? 0.05 : 0.0;

        return round($baseMonthlyRent * $multiplier, 2);
    }
}
```

Refactoring analysis. Extract Strategy removes the pressure to grow one quote method with plan-type conditionals. Introduce Parameter Object groups three correlated, validated inputs. New algorithms can implement `PricingStrategy`; existing clients continue calling `quote`. The cost is more classes and a runtime-selection responsibility.

Tests and evidence. The focused PHP runner checks base and combined dynamic results; the cross-language suite covers six shared cases per language. Commit: `7e9bd9ded113e91cca10e727a7eeff29c88d0ed8`.

Patch: `Part2/changes/CH01/dynamic_pricing.patch`. Pattern effect: Strategy newly introduced.

4.3 CH02 - Advanced property filtering

Requirement and acceptance criteria. Availability, case-insensitive city, inclusive rent range, amenity, and furnished rules must run alone or as an AND composite. The shared fixture must return only property id 1 for the five-rule composite. An inverted range must fail. Adding a rule must not require editing existing rules.

Before. `PropertyController::index` offers only a latest-first paginated listing. Adding several optional filters directly would create a growing conditional query-construction method.

After. Specification = `PropertySpecification`. CompositeSpecification = `AndSpecification`. Concrete specifications cover availability, city, rent range, amenity, furnishing, address, maximum rent, and status. `PropertyFilterService` evaluates the abstraction, and `PropertyController` composes request-driven address, rent, and status rules.

```
final readonly class AndSpecification implements PropertySpecification
{
    /** @param list<PropertySpecification> $specifications */
    public function __construct(private array $specifications)
    {
    }

    public function isSatisfiedBy(array $property): bool
    {
        foreach ($this->specifications as $specification) {
            if (! $specification->isSatisfiedBy($property)) {
                return false;
            }
        }

        return true;
    }
}
```

Refactoring analysis. Replace Conditional with Polymorphism moves independent predicates into named rule objects. Composite makes conjunction explicit and recursive. The Open/Closed benefit is practical: a new rule class can be combined without changing existing rule implementations. The trade-off is allocation and discoverability across more small classes; large data stores would translate specifications to database queries rather than filter every object in memory.

Tests and evidence. The PHP runner applies the shared five-rule fixture and returns only property id 1. Cross-language suites also check reusable single rules and invalid ranges. Commit: `2b2dd6246dd50f4a53a991e54`
Patch: `Part2/changes/CH02/advanced_filtering.patch`. Pattern effect: Specification newly introduced.

4.4 CH03 - Digital contract management

Requirement and acceptance criteria. A contract begins Draft. Only Draft can be sent. Sent can be signed or expired. Signed and Expired are terminal. Document and signature identities use 64-character lowercase SHA-256 hashes. An invalid hash or transition must fail.

Before. The public baseline has property occupancy and receipt records but no dedicated lease or digital-contract entity, signature evidence, or transition ownership.

After. State = `ContractState`. AbstractState = `AbstractContractState`, whose defaults reject all operations. Concrete states = `DraftState`, `SentState`, `SignedState`, and `ExpiredState`. Context = domain `DigitalContract`. A separate Eloquent `Models/DigitalContract` and migration persist property, tenant, hashes, and status.

```
    }

```

```
    public function documentHash(): string
    {
        return $this->documentHash;
    }

    public function signatureHash(): ?string
    {
        return $this->signatureHash;
    }

    public function send(): void
    {
        $this->state->send($this);
    }

    public function sign(string $signatureHash): void
    {
        self::assertSha256($signatureHash, 'Signature');
        $this->state->sign($this);
        $this->signatureHash = $signatureHash;
    }

    public function expire(): void
    {
        $this->state->expire($this);
    }

    public function transitionTo(ContractState $state): void
    {
        $this->state = $state;
    }

    private static function assertSha256(string $value, string $label): void
    {
        if (! preg_match('/^[a-f0-9]{64}$/', $value)) {
            throw new InvalidArgumentException("{$label} hash must be lowercase SHA-256.");
        }
    }
}
```

Refactoring analysis. Replace State-Dependent Conditional with State objects prevents a central status switch from expanding with every action. Each state owns allowed behavior; the abstract base makes denial the default. This improves transition locality but adds classes. The educational implementation stores integrity identifiers only. It does not provide signer identity, public-key validation, timestamping, non-repudiation, encryption, retention, or jurisdiction-specific e-signature compliance.

Tests and evidence. The PHP runner checks Draft → Sent → Signed, stored signature hash, blocked expiry of a signed contract, and invalid hash rejection. Commit: `8e2af082067bb83aab7e40caf2b7a463fef8b643`. Patch: `Part2/changes/CH03/digital_contract.patch`. Pattern effect: State newly introduced.

4.5 Before/after metrics and interpretation

Snapshot	Prod LOC	Test LOC	Tests	Types	Instances	Concrete
Original code	2055	27	1/2	5	5	18
After CH01	2143	27	2/2	6	6	21
After CH02	2303	27	4/4	7	7	32
After CH03	2450	120	10/10	8	8	38

Every LOC value is directly measured from the corresponding preserved local commit using the same rule: nonblank, noncomment physical PHP lines under `app/`, excluding tests and dependencies. The baseline's 1/2 is reported honestly: its stock feature test expects 200 while its root route intentionally redirects to login. Change-level acceptance counts cover the introduced behavior; the final snapshot also passes the corrected 2/2 Laravel suite. Improvement evidence is localized variation, composable

rules, explicit transitions, integration points, and passing acceptance behavior - not LOC growth by itself.

All five baseline collaborations remain. CH01 introduces the sixth type, Strategy; CH02 introduces the seventh, Specification; CH03 introduces the eighth, State, while extending Active Record with contract persistence. Final totals are eight types and eight instances. The complete version-by-version inventory is `data/patterns_evolution.csv`.

5 Part 3 - Cross-language analysis

5.1 Selected patterns and common specifications

Dynamic Rent Pricing (Strategy) and Advanced Property Filtering (Specification) were selected because both changes isolate volatile business rules and require composition/substitution across a stable client. State was not selected because the user request specifies two significant patterns.

Strategy fixture. Base rent is 1000. Base strategy returns 1000. Dynamic cases expect 1200 for peak only, 1100 for occupancy only, 1250 for peak plus occupancy minus long-stay discount, and 950 for long stay only. Month 13 must fail.

Specification fixture. Four properties vary by city, rent, availability, amenities, and furnishing. A composite of available + Kochi + rent 18000–25000 + parking + furnished returns only id 1. City alone returns three, available alone returns three, and an inverted range fails.

5.2 Actual results and static/dynamic comparison

Pattern	Language	LOC	Max CC	Runtime/compile	Median ms	Result
Strategy	Java	38	4	OpenJDK 17.0.20.1	64	6/6 passed
Strategy	Python	35	4	Python 3.12.3	43	6/6 passed
Strategy	JavaScript	33	4	Node.js 24.14.1	36	6/6 passed
Strategy	C++	42	4	G++ 13.3.0 C++20	11	6/6 passed
Specification	Java	33	2	OpenJDK 17.0.20.1	68	5/5 passed
Specification	Python	44	2	Python 3.12.3	46	5/5 passed
Specification	JavaScript	26	2	Node.js 24.14.1	37	5/5 passed
Specification	C++	62	2	G++ 13.3.0 C++20	11	5/5 passed

The timing protocol compiles first, performs one warmup, then measures five complete test processes on the same machine and reports the median. It includes startup and fixture assertions. The samples are preserved at `Part3/evidence/cross_language_runs.txt`; they do not support a claim that one language is generally faster.

5.3 Language comparison

Language	Abstraction	Composition / ownership	Observed trade-off

Java	Nominal interfaces and records	Strategy held by a service; Specification composite holds a list of interfaces.	Strong role visibility and immutable record fixtures; more declarations than JavaScript.
Python	Structural `Protocol` plus dataclasses	Ordinary object references; tuple-backed composite.	Concise immutable data and readable rules; interface conformance is primarily tooling/static-checker knowledge at authoring time.
JavaScript	Duck-typed classes	Strategy and rule objects share method shape without a nominal interface.	Smallest Specification LOC and direct composition; misspelled roles fail only when exercised.
C++	Abstract virtual base classes	`unique_ptr` for replaceable Strategy; `shared_ptr` for heterogeneous retained rules.	Ownership is explicit and startup here is small after compilation; templates or value erasure could reduce virtual allocation at higher complexity.

Despite syntax differences, the pattern structure is stable. Strategy keeps the context independent of a replaceable algorithm. Specification gives every predicate the same test operation and lets a composite hold other specifications. This structural equivalence, identical inputs, and identical outputs are stronger evidence than matching class names alone.

6 LLM records and human verification

Eight prompt records provide two records for each named team member. Each record has an exact prompt, a saved output, and a verification file. They cover baseline discovery/challenge, dynamic pricing/metrics, advanced filtering/language comparison, and digital contracts/dataset audit. The normalized index is `data/llm.csv`; files are under `prompts/`.

The records are labeled as OpenAI Codex draft runs and updated against the public baseline on 11 October 2026. They are technically checked against source, tests, and datasets, but they are not falsely presented as independently typed by each student. Every verification form therefore contains a pending member-acknowledgement line. The team must inspect and sign those records before submission.

Member	Technical scope	Automated verification	Share
Earlapally Sahith Siddarth	Public-source selection, architecture and baseline discovery	Exact baseline commit; five mapped rows	25%
Kankanala Sanithya Reddy	Dynamic pricing and Strategy metrics	Pricing checks; Strategy 6/6 in four languages	25%
Eati Manoj	Advanced filtering and cross-language Specification	Specification 5/5 in four languages	25%
Bussa Surya	Digital contracts, dataset audit, final integration	Contract checks; 2/2 Laravel suite; CSV reconciliation	25%

Human verification remains a mandatory final activity. `HUMAN_VERIFICATION.md` provides a team declaration and one signature/date field per member. Automated execution cannot verify authorship or institutional approval.

7 Dataset, metrics, and audit trail

The workbook `data/B4_Dataset.xlsx` mirrors seven authoritative CSV files. Its Summary sheet uses formulas to show five baseline instances, eight final instances, eight final types, 2,450 final production LOC, 395 added production LOC, 10 focused evolution checks, and 100 percent contribution allocation. Separate sheets preserve Patterns, Pattern Counts, Evolution, Cross Language, LLM Records, Metrics, and Contributions.

The CSV files use UTF-8, one header row, relative paths from `B4/`, and quoted comma-containing fields. The static metric script is `tools/metrics.py`. Source and test output are preserved as raw evidence rather than copied only into prose.

8 Findings and limitations

1. The most defensible baseline patterns are collaborations visible in the recorded public source. Rejecting Repository, Strategy, and Observer at baseline avoids promoting framework vocabulary or capability traits into unsupported instances.
2. Framework and application participants coexist. MVC, Active Record, middleware Chain of Responsibility, Form Request Template Method, and Facade explain the original Laravel design; Strategy, Specification, and State explain the introduced variation points.
3. The three changes increase class count deliberately. The benefit is that new pricing rules, search predicates, and contract states have named extension points and localized tests.
4. Cross-language syntax changes, but participant responsibilities and fixtures can remain constant. This makes pattern-level comparison more meaningful than raw LOC alone.
5. The evolved code is an academic change set, not a production deployment. Digital-contract evidence is hash-based and does not provide identity proof, PKI validation, trusted timestamps, non-repudiation, retention policy, or jurisdiction-specific compliance.
6. Timing results describe this machine and startup-heavy micro-tests. They do not rank languages generally.

9 Submission readiness

The deliverable contains one B4 team folder with one report, source snapshots, diagrams, tests, prompts, CSVs, and one Excel workbook. Before opening the pull request, the team must:

1. obtain and record faculty approval for the selected public application;
2. sign the central and per-prompt verification forms;
3. upload only `B4/` to the team's fork of the course repository;
4. commit the folder and record the final submission commit;
5. open one pull request titled **B4: Final submission** to the original repository's `main` branch and record its URL.

10 References

1. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
2. Mohsin Rafique, *Property Management*, https://github.com/mohsin-rafique/property-management, commit `f127b4e218018715586a52cede08e182ca45c983`, MIT License, accessed 2026-10-11.
3. Laravel 12 Documentation, https://laravel.com/docs/12.x, accessed 2026-10-11.
4. Laravel 12 Eloquent ORM, https://laravel.com/docs/12.x/eloquent, accessed 2026-10-11.
5. Laravel 12 Middleware, https://laravel.com/docs/12.x/middleware, accessed 2026-10-11.
6. PHP Manual, Object Interfaces, https://www.php.net/manual/en/language.oop5.interfaces.php, accessed 2026-10-11.
7. Team B4 changes, tests, raw timings, SHA-256 manifests, and normalized datasets included with this report.

A Evidence index

Path	Purpose
`Part1/evidence/test_results.txt`	Unchanged public baseline result: 1 passed, 1 stale example expectation failed.
`Part2/evidence/test_results.txt`	Evolved Laravel result: 2/2; focused evolution result: 10/10.
`Part3/evidence/cross_language_runs.txt`	Five timing samples and result output for eight implementations.
`Part1/evidence/source_manifest.sha256`	Integrity manifest for the baseline snapshot.
`Part2/evidence/source_manifest.sha256`	Integrity manifest for the evolved snapshot.
`Part2/changes/CH01..CH03`	Acceptance notes and unified source patches.
`data/patterns.csv`	Baseline pattern-to-source mapping.
`data/patterns_evolution.csv`	Per-snapshot status and participant evolution.
`data/cross_language.csv`	LOC, CC, runtime, command, result, median, and language notes.
`data/llm.csv`	Exact prompt/input/output/evaluation/verification index.
`data/B4_Dataset.xlsx`	Formatted Excel view of all normalized data.
`HUMAN_VERIFICATION.md`	Required final human sign-off.

B Folder structure

```
B4/
  README.md
  HUMAN_VERIFICATION.md
  B4_Report.tex
  B4_Report.pdf
  data/
    patterns.csv, pattern_counts.csv, patterns_evolution.csv
    cross_language.csv, llm.csv, metrics.csv, contributions.csv
    B4_Dataset.xlsx
  Part1/ code/, diagrams/, evidence/, llm/
  Part2/ code/, changes/CH01..CH03/, diagrams/, evidence/, llm/
  Part3/ strategy/, filter_specification/, evidence/, llm/, run_all.sh
  prompts/ two prompt-output-verification records per member
  tools/ metrics, diagram, and workbook builders
```